

Labwork 3: Hello, CUDA!

Philippe Olivier Schriqui-Duque
ID: 2640042

Introduction

The goal of this labwork is to convert an RGB image to gray, first with the CPU and then with the GPU using Numba CUDA, and to compare the time of both. I used a 4K image (3840×2880 , so about 11 million pixels).

Implementation

I load the image with `plt.imread` and flatten it into a 1D array of RGB pixels, so each row is one pixel with its 3 values:

```
img = plt.imread("img.jpg")
h, w, _ = img.shape
pixelCount = h * w
flat = img.reshape((pixelCount, 3))
```

For the gray value I use the simple formula from the course, the average of the 3 channels:

$$g = \frac{R + G + B}{3}$$

I convert the values to `int` before adding them, because they are `uint8` and the sum can go over 255.

CPU

On the CPU I just loop over all the pixels with a `for` loop and write the gray value in the 3 channels:

```
for i in range(pixelCount):
    g = (int(flat[i, 0]) + int(flat[i, 1]) + int(flat[i, 2])) // 3
    gray_cpu[i, 0] = g
    gray_cpu[i, 1] = g
    gray_cpu[i, 2] = g
```

GPU

On the GPU I write a kernel with `@cuda.jit`. Each thread does one pixel. The thread finds its pixel with its thread index, block index and block size. The `if` is needed because the last block can have more threads than pixels left.

```
@cuda.jit
def grayscale(src, dst):
    tid = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    if tid < src.shape[0]:
        g = (int(src[tid, 0]) + int(src[tid, 1]) + int(src[tid, 2])) // 3
```

```

dst[tidx, 0] = g
dst[tidx, 1] = g
dst[tidx, 2] = g

```

Then I follow the steps of the course: send the image to the GPU, run the kernel, and copy the result back to the CPU. The number of blocks is rounded up so all the pixels are covered.

```

def gray_gpu(blockSize):
    gridSize = (pixelCount + blockSize - 1) // blockSize
    devSrc = cuda.to_device(flat)
    devDst = cuda.device_array((pixelCount, 3), np.uint8)
    grayscale[gridSize, blockSize](devSrc, devDst)
    return devDst.copy_to_host()

```

The first call of the kernel is slow because Numba compiles it, so I run it once before measuring. I save both results with `plt.imsave` and I checked that the CPU and GPU images are the same.

Results

```

CPU time: 17.89 s
GPU time: 0.033 s
speedup: 543 x

```

The GPU is about 543 times faster than the CPU on this image.

Block size vs time

I tried different block sizes: 32, 64, 128, 256, 512 and 1024. For each one I ran the GPU version 20 times and took the average time.

```

block size 32    : 24.06 ms
block size 64    : 22.19 ms
block size 128   : 22.54 ms
block size 256   : 22.56 ms
block size 512   : 21.83 ms
block size 1024  : 21.64 ms

```

The graph of block size vs time is in the file `blocksize-vs-time.png`.

The graph is almost flat, all the block sizes take around 22 ms. This is because my time includes the copy of the image from the CPU to the GPU and back, and this copy takes most of the time. The computation itself is very simple (an average of 3 values per pixel), so changing the block size does not change much. The small differences between the block sizes also change from one run to another, so they are mostly noise.

Conclusion

The GPU version is much faster than the CPU version, because all the pixels are computed at the same time instead of one by one. For this simple operation the block size does not really matter, the transfer of the data between the CPU and the GPU is what takes the most time.